

Lecture 8

Recap: What is Maven Central?: it's a remote repository where the default most used dependencies and plugins for maven projects that we add to the pom.xml during the maven lifecycle exists.

Test Techniques and Strategies:

We have 3 types of testing techniques.

Black box: we design testing based on requirements and specifications no access to the actual code. E.g.: boundary value analysis. And equivalent partitioning.

White box: we design the tests based on the knowledge of the inner code.

experience based: Experience of testers, developers, users, ... about test object and environment. E.g.: Experience on where errors occurred in the past or are likely to occur in the future

What are advantages / disadvantages of each testing technique?

White box Testing:

Advantages:

Thorough coverage of code paths (conditions, loops, branches)

Helps find hidden logic errors and security issues early

Disadvantages:

Requires strong programming knowledge

Can miss missing requirements (you test what exists, not what should exist)

Black Box Testing:

Advantages:

Focuses on user requirements and behavior

Good at finding missing functionality

Disadvantages:

Inefficient for complex logic

Difficult to design test cases without insight into code

Experience Based Testing:

Advantages:

Excellent at finding edge cases and real-world bugs

Very effective when requirements are incomplete or changing

Disadvantages:

Highly dependent on tester's skill level

Hard to reproduce and measure

Equivalence Partitioning:

A black-box test technique in which the test cases are designed with regard to the execution of equivalence partitions, whereby one representative of each equivalence class is used.

A black box test where we divide the test values into partitions each represent many cases that results in the same test result. And we use one value per each partition to test these cases instead of using all the values.

Basic Idea:

- 1) Divide value ranges of inputs/outputs into equivalence partitions (EP)
- 2) All values of a partition show equivalent behavior of the test object
- 3) Meaningful sample: only one test value per EP (representative) If one value of the EP reveals an error, it is hoped that every other value of the EC will also reveal this error ,If a value of the EP does not detect an error, it is hoped that no other value of the EC will detect an error either.

Equivalence Partitioning – Examples Notes:

For each boundary specified value we have one valid group and 2 invalid group this includes false data types. So if we have a boundary between 1 and 100 a valid value would be from 2 till 99 an invalid would be 1 or less or 100 or more. And invalid group would be any data type that is not int.

Note on Testability: Don't give your users the option of entering wrong data

Defining Test Cases – Equivalence Partitioning:

For a test case to be considered complete we should provide all the required inputs.

We then should include all possible combinations that represents each group.

Minimizing Number of Test Cases:

At minimum we should include a value from each partition in at least one test case.

Once you've identified all representative values, combine them into test cases. Then rank those test cases based on how often users actually use them

Execute test cases starting from highest priority.

Don't waste effort on, Theoretically possible, but practically irrelevant test cases.

Among valid test cases, prefer ones that use boundary values, or combine multiple boundary values.

Inputs can interact so you should test: How values from different parameters behave together, Not just individually.

Pairwise: every pair of inputs is tested together at least once.

Do not combine representatives of invalid equivalence classes with representatives of other invalid equivalence classes: then it would be very hard to identify where the problem is actually at.

We end the equivalence partitioning testing when we reach 80% of test cases have been tested.

Equivalence Partitioning – Advantages and Disadvantages:

Advantages:

Number of test cases smaller compared to unsystematic debugging

Suitable for programs with many different input and output conditions

Disadvantages:

Considers conditions for individual input or output parameters

Consideration of interactions and dependencies of conditions very time consuming

Recommendation: To select effective test data: Combination of EPs with error-oriented methods, e.g. boundary value analysis

Boundary Value Analysis:

Boundary Value Analysis A black-box test technique in which test cases are designed based on boundary values. [After ISO 29119, Glossary ITSQB V3.3]

Why BVA: Bugs often occur at edges of conditions, like loops or branches. “Off-by-one” errors are common here.

Test data at boundaries finds failures more reliably than random or middle values.

It’s the best practice to use equivalence partitioning with boundary value analysis for the best results.

How would boundary value analysis and equivalent partitioning work together, Test the limits of the EP: smallest and largest value per EP Each boundary of an EP must occur in a test data combination

We should include valid boundary and invalid boundary values.

We should also include the max and minimum, below and above those values this then tests our data structures.

We end the boundary test analysis when we reach 90% of test cases being done.

Boundary Value Analysis – Advantages and Disadvantages:

Advantages:

Errors are more likely to be found at the boundaries of equivalence partitions than within
Boundary value analysis is one of the most useful methods for test case design when used correctly

Efficient combination with other methods leave degrees of freedom in choice of test data

Disadvantages:

Difficult to specify clear-cut recipes for selection of test data

Difficult to determine all relevant boundary values

Creativity required to find successful test data

Often not applied efficiently enough as it appears too simple

Decision Table Testing:

Prerequisite: Use DTT when requirements have logical conditions and complex rules.

Procedure

1. Examine the specification – understand what the system should do.
2. Determine input conditions and actions – figure out which inputs affect which outputs.
3. Ensure conditions are binary – each condition should be true/false (or yes/no) for building the table.

The format of the table:

Conditions **Rules**

Actions **Action associations**

Conditions: which condition is the system in (object status)

Rules: is this condition true or false for each condition

Actions: what will be the systems response to the summation of different status; outputs the system produces for that combination.

Action Assocciations: they show which actions happen for each rule.

Decision Table Test – Advantages and Disadvantages:

Advantages:

Combinations of conditions that might otherwise remain untested

Decision table technique generally suitable if software processes depend on several logical decisions

Logical relationships can be systematically formulated

Decision tables can be easily checked for redundancy, consistency and completeness

Disadvantages:

Difficult to understand if there are too many conditions

Relationships between individual conditions can only be expressed implicitly (cf. cause-effect graph analysis)